
Dev d'appli mobile pour les nuls

De Nul à Expert

Basé sur le projet Social Seller

Sprint 1

Expo + React Native

Supabase + RLS

Ce livre t'apprend à construire une vraie application mobile SaaS de A à Z, en suivant le développement réel de Social Seller — une plateforme pour les commerçants africains. Chaque concept est expliqué avec des analogies, du code réel, des quiz, et des astuces de pro.

■ Configurer un projet Expo/React Native

■ Créer des screens d'authentification

■ Supabase : base de données + auth + RLS

■ Navigation avec Expo Router v4

■ TypeScript pour les nuls

■ Sécuriser une app mobile

■ Git et GitHub en pratique

■ Débugger comme un pro

Mohamed · Social Seller Project · 2026

Table des matières

PARTIE 1 — Les bases

- | | |
|-------------------|---|
| Chapitre 1 | Bienvenue dans le dev mobile — comprendre l'univers |
| Chapitre 2 | Le stack technique de Social Seller — pourquoi ces outils ? |
| Chapitre 3 | Installer et configurer l'environnement de développement |
| Chapitre 4 | TypeScript pour les nuls — écrire du code qui ne plante pas |

PARTIE 2 — Sprint 1 en profondeur

- | | |
|--------------------|---|
| Chapitre 5 | Structure du projet Expo Router v4 — naviguer dans les fichiers |
| Chapitre 6 | L'écran Login — ton premier screen React Native |
| Chapitre 7 | S'inscrire — valider des données et créer un compte |
| Chapitre 8 | Supabase Auth — l'authentification sans serveur |
| Chapitre 9 | Activation WhatsApp — UX et états de l'interface |
| Chapitre 10 | Profile Setup — formulaires avancés et upload d'image |
| Chapitre 11 | Supabase : base de données, migrations et RLS |
| Chapitre 12 | Row Level Security — sécuriser ses données côté serveur |
| Chapitre 13 | Navigation avec Expo Router — tabs et authentification |

PARTIE 3 — Bonnes pratiques

- | | |
|--------------------|--|
| Chapitre 14 | Sécurité mobile — les règles d'or |
| Chapitre 15 | Debugger comme un pro — lire les erreurs et les résoudre |
| Chapitre 16 | Git et GitHub — versionner son travail |
| Chapitre 17 | Récap Sprint 1 et préparation Sprint 2 |

PARTIE 1

Les bases

Comprendre l'univers du dev mobile avant de coder

- Bienvenue dans le dev mobile
 - Le stack technique
- Configurer l'environnement
- TypeScript pour les nuls

Chapitre 1 — Bienvenue dans le dev mobile

Comprendre l'univers du développement mobile

Tu as une idée d'appli. Super ! Mais par où commencer ? Dans ce chapitre, on démystifie le monde du développement mobile et on t'explique pourquoi c'est accessible même si tu pars de zéro.

■ Analogie : L'application, c'est comme une maison

Une maison a une structure (les murs = React Native), une décoration (les écrans = composants), une plomberie (les données = Supabase), et un accès sécurisé (la porte = authentification). Dans ce livre, tu vas construire cette maison de A à Z.

1.1 — Native vs Cross-Platform

Il existe deux grandes familles d'applications mobiles :

Type	Technologie	Avantage	Inconvénient
Native iOS	Swift/Obj-C	Performances maximales	Mac + Xcode obligatoires
Native Android	Kotlin/Java	Accès API Android complet	Seul Android
Cross-platform	React Native / Flutter	Un seul code = iOS + Android	Compromis de performance
Notre choix ■	React Native (Expo)	Un code → iOS + Android + Web	Quelques limitations natives

1.2 — Qu'est-ce qu'Expo ?

Expo est une surcouche de React Native qui simplifie tout. Sans Expo, configurer React Native prend 2 jours et nécessite Android Studio + Xcode. Avec Expo, tu es opérationnel en 10 minutes.

■ Expo Go vs Development Build

Expo Go est une app de test gratuite sur ton téléphone. Elle supporte uniquement les SDK récents. Pour un projet sérieux, on créera un 'Development Build' (Sprint 2) — une app installable sur ton téléphone qui supporte exactement ton SDK. Aucun Expo Go requis.

1.3 — Le projet Social Seller

Social Seller est une application SaaS multi-tenant pour les commerçants africains. Elle centralise les messages WhatsApp, TikTok et Facebook, gère les commandes et le stock. C'est le projet fil rouge de ce livre — chaque concept sera illustré avec du vrai code de cette app.

Frontend Mobile	Expo + React Native	Les écrans que l'utilisateur voit
Navigation	Expo Router v4	Passer d'un écran à l'autre
Base de données	Supabase (PostgreSQL)	Stocker les données
Authentification	Supabase Auth	Gérer les comptes utilisateurs
Sécurité	RLS (Row Level Security)	Protéger les données
Backend API	Node.js sur Railway	Logique métier serveur (Sprint 2)

Quiz : Pourquoi utilise-t-on React Native plutôt que Swift pour Social Seller ?

- Swift est plus rapide
- React Native permet un seul code pour iOS, Android et Web
- React Native est gratuit
- Swift ne supporte pas Supabase

Chapitre 2 — Le stack technique de Social Seller

Pourquoi ces outils et pas d'autres ?

Choisir son stack, c'est comme choisir ses outils avant de construire. Un mauvais choix en début de projet coûte cher plus tard. Voici pourquoi chaque technologie a été choisie pour Social Seller.

2.1 — Expo SDK 54 + React Native

React Native te permet d'écrire du JavaScript/TypeScript et d'obtenir une vraie app native (pas une webview). Expo ajoute une couche de confort : pas besoin de configurer Xcode ou Android Studio pour commencer.

■ ■ Pourquoi SDK 54 et pas 56 ?

Expo Go sur Android supporte actuellement SDK 54. SDK 56 vient de sortir et Expo Go n'est pas encore mis à jour. En production, on utilisera un Development Build qui supporte n'importe quel SDK — mais pour le dev, SDK 54 + Expo Go = zéro friction.

2.2 — TypeScript

TypeScript est JavaScript avec des types. Ça veut dire que l'éditeur de code te prévient AVANT que l'app plante. Dans un projet en équipe ou qui grandit, c'est indispensable.

```
// JavaScript – pas de types, erreur possible à l'exécution
const phone = getPhoneNumber()
phone.toUpperCase() // plante si phone est null !
// TypeScript – erreur détectée AVANT l'exécution
const phone: string | null = getPhoneNumber()
if (phone) {
  phone.toUpperCase() // OK, TypeScript sait que c'est une string ici
}
```

2.3 — Expo Router v4

Expo Router utilise le système de fichiers pour définir la navigation. Un fichier = un écran. Plus besoin de configurer manuellement un stack navigator — c'est automatique.

■ Analogie : Expo Router, c'est comme les pages web

Sur un site web, l'URL /login charge la page login. Avec Expo Router, le fichier app/(auth)/login.tsx correspond à l'écran login. L'URL et le fichier sont synchronisés — même logique, même simplicité.

2.4 — Supabase

Supabase est un Backend-as-a-Service (BaaS) open-source. Il te donne gratuitement : une base de données PostgreSQL, une authentification complète, du stockage de fichiers, et des APIs REST + temps réel. Sans Supabase, il faudrait coder tout ça soi-même.

Auth (Email, Phone...)	Système de login/logout, tokens JWT, refresh...
PostgreSQL	Serveur de base de données, backups, scaling...
Row Level Security	Vérification manuelle des permissions dans le code
Storage	Serveur de fichiers, CDN, upload/download
Realtime	WebSockets, synchronisation en temps réel

■ Supabase vs Firebase

Firebase (Google) est le concurrent direct. Supabase est open-source (tu peux l'héberger toi-même), utilise SQL standard (PostgreSQL), et a un tier gratuit très généreux. Pour un projet africain avec des contraintes de coûts, Supabase est souvent le meilleur choix.

Chapitre 3 — Installer et configurer l'environnement

De zéro à un projet qui tourne

Dans ce chapitre, on reprend exactement ce qu'on a fait pour Social Seller au démarrage. Chaque commande est expliquée — pas de magie noire.

3.1 — Ce dont tu as besoin

1 Node.js 20+

Le moteur JavaScript côté serveur. npm (gestionnaire de packages) est inclus.

2 VS Code

L'éditeur de code recommandé. Gratuit, avec l'extension React Native Tools.

3 Git

Pour versionner ton code. Installé par défaut sur Mac/Linux.

4 Compte Supabase

Gratuit sur supabase.com. Aucune carte bancaire requise.

5 Compte GitHub

Pour héberger ton code en privé. Gratuit.

6 Expo Go

L'app sur ton téléphone pour tester. Disponible sur App Store et Play Store.

3.2 — Créer le projet Expo

Une seule commande crée la structure complète du projet :

```
# Créer le projet Social Seller avec TypeScript + Expo Router
npx create-expo-app social-seller --template tabs

# Entrer dans le dossier
cd social-seller

# Lancer l'app en mode développement
npx expo start
```

■■ L'erreur --legacy-peer-deps

Expo Router a un conflit de dépendances avec react-dom. Si tu vois une erreur npm, ajoute --legacy-peer-deps à chaque commande npm install. Exemple : npx expo install expo-camera --legacy-peer-deps. Ce n'est pas un bug — c'est un problème connu résolu automatiquement dans les prochaines versions.

3.3 — Structure des fichiers créés

Voici la structure du projet Social Seller après le Sprint 1. Chaque dossier a un rôle précis :

```
social-seller/
  ■■■ app/ ← Tous les écrans (Expo Router)
  ■ ■■■ _layout.tsx ← Layout racine (Stack navigator)
  ■ ■■■ index.tsx ← Point d'entrée (redirect auth)
  ■ ■■■ (auth)/ ← Groupe "non authentifié"
  ■ ■ ■■■ _layout.tsx
  ■ ■ ■■■ login.tsx ← Écran de connexion
  ■ ■ ■■■ register.tsx ← Inscription
  ■ ■ ■■■ activation.tsx ← Activation WhatsApp
  ■ ■ ■■■ profile-setup.tsx ← Configuration du profil
  ■ ■■■ (tabs)/ ← Groupe "authentifié"
  ■ ■■■ _layout.tsx ← Tab bar (5 onglets)
  ■ ■■■ index.tsx ← Dashboard principal
  ■■■ lib/
  ■ ■■■ supabase.ts ← Client Supabase (singleton)
  ■ ■■■ constants.ts ← Constantes globales
  ■ ■■■ database.types.ts ← Types TypeScript auto-générés
  ■■■ supabase/
  ■ ■■■ migrations/ ← Migrations SQL versionnées
  ■ ■■■ 001_initial_schema.sql
  ■ ■■■ 002_...
  ■ ■■■ 003_tenant_profile_setup.sql
  ■ ■■■ 004_create_initial_tenant_rpc.sql
  ■■■ .env.local ← Variables secrètes (JAMAIS committé)
  ■■■ .gitignore
  ■■■ package.json
```

■ Le fichier .env.local est sacré

Ne committe JAMAIS .env.local sur GitHub. Il contient ta clé Supabase. Si elle est exposée, quelqu'un peut lire et modifier toute ta base de données. .gitignore doit toujours contenir .env.local — vérifie-le avant chaque commit.

Chapitre 4 — TypeScript pour les nuls

Écrire du code qui ne plante pas

TypeScript ajoute des types à JavaScript. Au lieu de découvrir les erreurs quand l'app plante sur le téléphone d'un client, tu les vois immédiatement dans ton éditeur. Voici les notions essentielles utilisées dans Social Seller.

4.1 — Les types de base

```
// Les types primitifs
const nom: string = "Mohamed"
const age: number = 30
const actif: boolean = true
const rien: null = null
const indefini: undefined = undefined

// TypeScript infère souvent le type automatiquement
const prenom = "Ali" // TypeScript sait que c'est une string

// Tableau
const numeros: string[] = ["+223700000001", "+223700000002"]

// Objet avec interface
interface Tenant {
  id: string
  name: string
  country: string
  owner_id: string
  created_at: string
}

// Usage
const shop: Tenant = {
  id: "uuid-123",
  name: "Boutique Ali",
  country: "ML",
  owner_id: "user-uuid",
  created_at: "2026-06-21T10:00:00Z"
}
```

4.2 — Les types optionnels et union

```
// Type optionnel (peut être null ou undefined)
interface ProfileSetup {
```

```

shopName: string
sector: string
country: string
logoUrl?: string // ← le ? veut dire "optionnel"
}

// Union de types (l'une ou l'autre valeur)
type LoadingState = 'idle' | 'loading' | 'success' | 'error'

// Usage dans un composant
const [state, setState] = useState<LoadingState>('idle')

// Type nullable explicite (pattern commun avec Supabase)
const [session, setSession] = useState<Session | null | undefined>(undefined)
// undefined = on ne sait pas encore (chargement)
// null = pas de session (non connecté)
// Session = connecté

```

4.3 — Les fonctions typées

```

// Fonction avec types d'entrée et de sortie
function buildEmail(phone: string): string {
  return phone + '@socialseller.app'
}

// Fonction asynchrone (retourne une Promise)
async function getUser(): Promise<User | null> {
  const { data, error } = await supabase.auth.getUser()
  if (error) return null
  return data.user
}

// Props d'un composant React Native
interface SelectFieldProps {
  label: string
  placeholder: string
  value: string
  options: string[]
  onChange: (value: string) => void // ← fonction comme prop
}

```

■ TypeScript n'alourdit pas le code

Au début, les types semblent une contrainte. En réalité, ils font gagner des heures de debug. VS Code utilise les types pour l'autocomplétion — tu n'as plus besoin de retenir tous les noms de fonctions. Tape 'supabase.auth.' et VS Code montre toutes les méthodes disponibles.

Quiz : Que signifie logoUrl?: string dans une interface TypeScript ?

- ☐ logoUrl est une string obligatoire
- ☒ logoUrl peut être une string ou undefined
- ☐ logoUrl est un tableau de strings
- ☐ logoUrl ne peut pas être une string

PARTIE 2

Sprint 1 en profondeur

Décortiquer chaque ligne de code de Social Seller

- Structure Expo Router v4
 - Écran Login
- Register et validation
 - Supabase Auth
- Activation WhatsApp
 - Profile Setup
- Migrations et triggers
 - RLS
- Navigation tabs

Chapitre 5 — Structure du projet Expo Router v4

Naviguer dans les fichiers comme un pro

Expo Router v4 utilise le système de fichiers pour définir la navigation. La structure des dossiers devient automatiquement la structure des écrans et des URLs. Comprendre cette logique est fondamental pour tout le reste.

5.1 — La convention de nommage

app/index.tsx	/	Page d'accueil / redirect
app/(auth)/login.tsx	/({auth})/login	Écran de login
app/(tabs)/index.tsx	/({tabs})	Premier onglet
app/(tabs)/inbox.tsx	/({tabs})/inbox	Onglet messages
app/_layout.tsx	(layout racine)	S'applique à TOUTES les routes
app/(auth)/_layout.tsx	(layout groupe auth)	S'applique au groupe (auth)

■ Analogie : Les groupes (auth) et (tabs), c'est comme des dossiers invisibles

Les parenthèses signifient que le nom du groupe n'apparaît PAS dans l'URL. Le fichier `app/(auth)/login.tsx` donne la route `/login`, pas `/({auth})/login`. C'est juste une façon d'organiser les fichiers par contexte (auth vs tabs) sans polluer les URLs.

5.2 — Le pattern auth dans app/index.tsx

Au lieu de gérer la navigation auth dans `_layout.tsx` (ce qui cause des bugs), Social Seller utilise un `app/index.tsx` qui redirige selon l'état de session :

```
// app/index.tsx – Point d'entrée auth
import { Redirect } from 'expo-router'
import { useEffect, useState } from 'react'
```

```

import { View, Text } from 'react-native'
import { supabase } from '../lib/supabase'
import type { Session } from '@supabase/supabase-js'

export default function Index() {
  // undefined = on charge | null = pas connecté | Session = connecté
  const [session, setSession] = useState<Session | null | undefined>(undefined)

  useEffect(() => {
    supabase.auth.getSession()
      .then(({ data: { session } }) => setSession(session ?? null))
      .catch(() => setSession(null))
  }, [])

  // Pendant le chargement → écran violet
  if (session === undefined) {
    return (
      <View style={{ flex: 1, backgroundColor: '#6366F1',
        justifyContent: 'center', alignItems: 'center' }}>
        <Text style={{ color: 'white', fontSize: 18 }}>Chargement...</Text>
      </View>
    )
  }

  // Redirect est un composant Expo Router — plus fiable que router.replace()
  return <Redirect href={session ? '/(tabs)' : '/(auth)/login'} />
}

```

■ Leçon clé : Redirect vs router.replace()

Ne jamais appeler `router.replace()` dans un `useEffect` pendant que le Stack navigator n'est pas encore monté — la navigation est ignorée silencieusement. Utilise le composant d'Expo Router : il s'exécute pendant le render, quand le navigateur est prêt.

Chapitre 6 — L'écran Login

Ton premier screen React Native

Le Login est l'écran le plus vu par tes utilisateurs. Il doit être rapide, clair, et sécurisé. Dans ce chapitre, on décortique chaque ligne de code de login.tsx.

6.1 — Anatomie d'un composant React Native

```
// app/(auth)/login.tsx – Structure de base
import { useState } from 'react'
import { View, Text, TextInput, TouchableOpacity,
  ActivityIndicator, StyleSheet } from 'react-native'
import { useRouter } from 'expo-router'
import { supabase } from '../../lib/supabase'
import { COUNTRY_CODE, EMAIL_DOMAIN, PHONE_DIGITS_LENGTH } from
  '../../lib/constants'

// ① Constantes sécurité – JAMAIS de console.log avec ces valeurs
const EMAIL_SUFFIX = EMAIL_DOMAIN // '@socialseller.app'
export default function LoginScreen() {
  // ② État local du composant
  const [digits, setDigits] = useState('') // les 8 chiffres du téléphone
  const [password, setPassword] = useState('')
  const [showPassword, setShowPassword] = useState(false)
  const [isLoading, setIsLoading] = useState(false)
  const [errorMessage, setErrorMessage] = useState('')
  const router = useRouter()

  // ③ La logique métier
  const handleLogin = async () => { /* ... voir section 6.2 */ }

  // ④ Le rendu visuel
  return (
    <View style={styles.container}>
      { /* ... composants visuels */ }
    </View>
  )
}

// ⑤ Les styles (comme du CSS)
const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#F8FAFC', padding: 24 },
  // ...
})
```

6.2 — L'astuce d'auth sans Twilio

Social Seller authentifie les utilisateurs par numéro de téléphone. Normalement, ça nécessite Twilio (payant, ~\$0.05 par SMS). Notre astuce : on utilise le provider Email de Supabase avec un email construit :

■ Analogie : L'email construit — transformer un téléphone en email

Supabase n'accepte que les emails, pas les téléphones directement (sans Twilio). Alors on transforme +22370123456 en +22370123456@socialseller.app. Cet email est fictif mais unique pour chaque utilisateur. Supabase le traite comme un vrai email — sans envoyer de SMS. Gratuit et sécurisé.

```
// lib/constants.ts – Les constantes centralisées
export const EMAIL_DOMAIN = '@socialseller.app'
export const COUNTRY_CODE = '+223' // Préfixe Mali
export const PHONE_DIGITS_LENGTH = 8 // 8 chiffres après +223

// Construction de l'email dans login.tsx
const handleLogin = async () => {
  if (digits.length !== PHONE_DIGITS_LENGTH) {
    setErrorMessage(`Entrez ${PHONE_DIGITS_LENGTH} chiffres`)
    return
  }
  setIsLoading(true)
  setErrorMessage('')

  // ← L'astuce : construire l'email depuis le numéro
  const email = COUNTRY_CODE + digits + EMAIL_DOMAIN
  // ex: +22370123456@socialseller.app

  const { error } = await supabase.auth.signInWithPassword({ email, password })
  if (error) {
    setErrorMessage(error.message)
  } else {
    router.replace('/(tabs)')
  }
  setIsLoading(false)
}
```

■ Jamais de console.log avec des données sensibles

Ne jamais écrire console.log('email:', email) ou console.log('password:', password). Les logs apparaissent dans les outils de debug et peuvent être lus par des tiers. Cette règle est dans CLAUDE.md — elle s'applique à tout le projet.

6.3 — Les composants React Native utilisés

View	div	Conteneur principal, sections
Text	p, h1, span	Labels, titre, messages d'erreur
TextInput	input	Champ téléphone, champ mot de passe
TouchableOpacity	button	Bouton Se connecter
ActivityIndicator	spinner CSS	Loading pendant l'auth
KeyboardAvoidingView	(pas d'équivalent)	Empêche le clavier de cacher les champs

Chapitre 7 — S'inscrire et valider des données

Créer un compte et valider les inputs

L'écran Register est plus complexe que Login : prénom, nom, téléphone, mot de passe, confirmation. Chaque champ doit être validé AVANT d'envoyer quoi que ce soit à Supabase. La validation côté client protège l'UX, mais ne remplace jamais la validation serveur.

7.1 — La validation en cascade

```
// La validation doit être dans l'ordre logique
const handleRegister = async () => {
  // Étape 1 – Noms
  if (!firstName.trim() || !lastName.trim()) {
    setErrorMessage('Prénom et nom requis')
    return // ← Sortie immédiate, on n'continue pas
  }

  // Étape 2 – Téléphone
  if (digits.length !== PHONE_DIGITS_LENGTH) {
    setErrorMessage(`Numéro à ${PHONE_DIGITS_LENGTH} chiffres requis`)
    return
  }

  // Étape 3 – Mot de passe
  if (password.length < 8) {
    setErrorMessage('Mot de passe : 8 caractères minimum')
    return
  }

  // Étape 4 – Confirmation
  if (password !== confirmPassword) {
    setErrorMessage('Les mots de passe ne correspondent pas')
    return
  }

  // Tout est valide → on appelle Supabase
  setIsLoading(true)
  const email = COUNTRY_CODE + digits + EMAIL_DOMAIN
  const { error } = await supabase.auth.signUp({
    email,
    password,
    options: {
      data: {
```

```
first_name: firstName.trim(),
last_name: lastName.trim(),
phone: COUNTRY_CODE + digits, // stocké dans les métadonnées user
}
}
})
if (error) {
  setErrorMessage(error.message)
} else {
  router.replace('/(auth)/activation') // → écran WhatsApp
}
setIsLoading(false)
}
```

■ options.data dans signUp()

Supabase stocke les données supplémentaires dans `auth.users.raw_user_meta_data`. C'est là qu'on met `first_name`, `last_name`, `phone`. Notre trigger SQL lit ces données pour créer la ligne dans `public.users`. C'est le seul endroit sûr pour passer des données personnelles à l'inscription.

7.2 — Le rate limit Supabase

Sur le tier gratuit de Supabase, les appels d'inscription sont limités à environ 30/heure. En développement, quand on teste beaucoup, on atteint vite cette limite.

■■ Solution : SMTP custom (Resend ou Postmark)

Configurez un provider SMTP dans Supabase Dashboard → Authentication → SMTP Settings. Resend.com et Postmark offrent des tiers gratuits généreux. Une fois configuré, le rate limit Supabase ne s'applique plus — vous utilisez votre propre quota email. À faire au début du Sprint 2.

Chapitre 8 — Supabase Auth en profondeur

Comprendre l'authentification sans serveur

Supabase Auth gère tout : création de compte, connexion, tokens JWT, refresh automatique. Dans ce chapitre, on comprend ce qui se passe sous le capot.

8.1 — Les tokens JWT

■ Analogie : Le JWT, c'est comme un badge d'accès

Quand tu te connectes à un immeuble, le gardien te donne un badge (le JWT). Ce badge contient ton identité et tes droits. Pour chaque porte (chaque requête API), tu montres ton badge — le serveur vérifie qu'il est valide sans appeler le gardien à chaque fois. Le badge expire après 1 heure (access_token) — un token de renouvellement (refresh_token) permet d'en obtenir un nouveau automatiquement.

```
// Ce que Supabase stocke après signInWithPassword
{
  "access_token": "eyJhbGciOiJIUzI1NiJ9...", // JWT valide 1h
  "refresh_token": "abc123xyz...", // Valide 30 jours
  "expires_at": 1719000000, // Timestamp d'expiration
  "user": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "email": "+22370123456@socialseller.app",
    "raw_user_meta_data": {
      "first_name": "Mohamed",
      "last_name": "Fomba",
      "phone": "+22370123456"
    }
  }
}
```

8.2 — Le client Supabase dans lib/supabase.ts

```
// lib/supabase.ts — Le singleton Supabase
import { Platform } from 'react-native'
import * as SecureStore from 'expo-secure-store'
import { createClient } from '@supabase/supabase-js'
import type { Database } from '../database.types'

// Adapter storage platform-aware
```

```

const storage = {
  getItem: (key: string) =>
    Platform.OS === 'web'
    ? Promise.resolve(localStorage.getItem(key)) // web = localStorage
    : SecureStore.getItemAsync(key), // native = SecureStore
  setItem: (key: string, value: string) =>
    Platform.OS === 'web'
    ? Promise.resolve(localStorage.setItem(key, value))
    : SecureStore.setItemAsync(key, value),
  removeItem: (key: string) =>
    Platform.OS === 'web'
    ? Promise.resolve(localStorage.removeItem(key))
    : SecureStore.deleteItemAsync(key),
}

// Variables depuis .env.local (jamais hardcodées)
const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL!
const supabaseAnonKey = process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY!
export const supabase = createClient<Database>(supabaseUrl, supabaseAnonKey, {
  auth: {
    storage, // ← notre adapter custom
    autoRefreshToken: true, // ← renouvelle le JWT automatiquement
    persistSession: true, // ← garde la session entre les lancements
    detectSessionInUrl: false, // ← désactivé (pas d'OAuth web)
  },
})

```

■ expo-secure-store vs AsyncStorage

AsyncStorage stocke les données en clair dans le système de fichiers. N'importe quelle app malveillante peut lire ces données. expo-secure-store utilise le Keychain iOS / Keystore Android — chiffrés par l'OS. Pour les tokens JWT, TOUJOURS expo-secure-store.

8.3 — Les méthodes auth les plus utilisées

signUp()	Créer un compte	Écran Register
signInWithPassword()	Se connecter	Écran Login
signOut()	Se déconnecter	Menu Profil

getSession()	Lire la session depuis le storage	Au démarrage de l'app
getUser()	Valider le JWT avec le serveur	Avant une opération sensible
onAuthStateChange()	Écouter les changements d'auth	Pour réagir au login/logout

Chapitre 9 — Activation WhatsApp et états UI

Gérer les états d'interface complexes

L'écran Activation est un excellent exemple de gestion d'états multiples : l'utilisateur envoie un lien, attend, peut renvoyer, et peut skip. Chaque action change l'interface différemment.

9.1 — La machine à états de l'activation

Initial	isSent=false, countdown=0	Champ téléphone + bouton Envoyer
Loading	isLoading=true	Spinner sur le bouton
Envoyé	isSent=true, countdown=58	Message de succès + compte à rebours
Countdown fini	isSent=true, countdown=0	Bouton Renvoyer actif
Erreur	errorMessage != ""	Message d'erreur en rouge

9.2 — Le compte à rebours avec setInterval

```
// Compte à rebours de 58 secondes après envoi du lien
const RESEND_DELAY_SECONDS = 58
const handleSend = async () => {
  // ... appel API ...
  // Démarrer le compte à rebours
  setIsSent(true)
  setCountdown(RESEND_DELAY_SECONDS)
  const interval = setInterval(() => {
    setCountdown(prev => {
      if (prev <= 1) {
        clearInterval(interval) // ← Arrêter quand on arrive à 0
        return 0
      }
      return prev - 1
    })
  })
}
```



```

}, 1000) // ← Toutes les 1000ms (1 seconde)
}

// Dans le rendu :
{countdown > 0 ? (
<Text>Renvoyer dans {countdown}s</Text>
) : (
<TouchableOpacity onPress={handleSend}>
<Text>Renvoyer</Text>
</TouchableOpacity>
)}

```

■ ■ Toujours nettoyer les intervals

Si le composant se démonte (l'utilisateur navigue ailleurs) pendant que l'interval tourne, il continue à appeler `setCountdown` sur un composant inexistant → memory leak. Solution : retourner une fonction de cleanup dans `useEffect` qui appelle `clearInterval`.

9.3 — La gestion du backend absent

En Sprint 1, le backend Railway n'est pas encore déployé. L'écran gère ce cas gracieusement :

```

// Si l'URL du backend n'est pas configurée → message clair
const apiUrl = process.env.EXPO_PUBLIC_API_URL
if (!apiUrl) {
  setErrorMessage('Service temporairement indisponible')
  setIsLoading(false)
  return // ← Ne pas faire de fetch qui planterait
}

// L'URL est là → on peut appeler l'API
const response = await fetch(`${apiUrl}/auth/whatsapp-activation`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ phone: COUNTRY_CODE + digits }),
})

```

Chapitre 10 — Profile Setup et formulaires avancés

Formulaires multi-plateformes et upload de logo

Profile Setup est l'écran le plus technique du Sprint 1 : sélecteur natif iOS, Picker Android/Web, upload d'image, et insert en base de données via RPC sécurisé.

10.1 — Les sélecteurs platform-aware

iOS a `ActionSheetIOS` (une modale native en bas d'écran). Android et Web n'ont pas cet équivalent — on utilise `@react-native-picker/picker`. Le code doit adapter l'UI selon la plateforme :

```
// Composant SelectField – adaptatif selon la plateforme
import { Platform } from 'react-native'
import { Picker } from '@react-native-picker/picker'

function SelectField({ label, value, options, onChange }: SelectFieldProps) {
  // iOS → ActionSheetIOS (menu natif)
  const openActionSheet = () => {
    if (Platform.OS === 'ios') {
      ActionSheetIOS.showActionSheetWithOptions(
        { options: [...options, 'Annuler'], cancelButtonIndex: options.length },
        (buttonIndex) => {
          if (buttonIndex < options.length) onChange(options[buttonIndex])
        }
      )
    }
  }

  // iOS → Pressable qui ouvre l'ActionSheet
  if (Platform.OS === 'ios') {
    return (
      <Pressable onPress={openActionSheet}>
        <Text>{value || label}</Text>
      </Pressable>
    )
  }

  // Android + Web → Picker inline
  return (
    <View style={{ borderWidth: 1, borderColor: '#E2E8F0', borderRadius: 8 }}>
      <Picker selectedValue={value} onValueChange={onChange}>
        {options.map(opt => <Picker.Item key={opt} label={opt} value={opt} />)}
      </Picker>
    </View>
  )
}
```

```
</Picker>
</View>
)
}
```

■■ Platform.OS : 'ios' | 'android' | 'web'

Platform.OS est la façon standard de détecter la plateforme dans React Native. Attention : la condition doit être === 'ios' (pas !== 'android') pour inclure le web dans la branche Picker. Cette erreur a causé un bug en production dans notre projet !

10.2 — L'upload d'image avec expo-image-picker

```
// Sélectionner une image depuis la galerie
import * as ImagePicker from 'expo-image-picker'

const pickImage = async () => {
  // Demander la permission d'accès à la galerie
  const { status } = await ImagePicker.requestMediaLibraryPermissionsAsync()
  if (status !== 'granted') {
    alert('Permission requise pour accéder aux photos')
    return
  }
  // Ouvrir le sélecteur
  const result = await ImagePicker.launchImageLibraryAsync({
    mediaTypes: ImagePicker.MediaTypeOptions.Images,
    allowsEditing: true,
    aspect: [1, 1], // ← carré pour un logo
    quality: 0.8, // ← compression 80%
  })
  if (!result.canceled) {
    setLogoUri(result.assets[0].uri)
  }
}

// Upload vers Supabase Storage
const uploadLogo = async (uri: string, tenantId: string) => {
  const ext = uri.split('.').pop()
  const path = `tenants/${tenantId}/logo.${ext}`
  // Lire le fichier comme ArrayBuffer
  const response = await fetch(uri)
  const blob = await response.blob()
  const { error } = await supabase.storage
    .from('logos')
    .upload(path, blob, { upsert: true })
}
```

```
if (!error) {  
  const { data } = supabase.storage.from('logos').getPublicUrl(path)  
  return data.publicUrl  
}  
return null // ← échec non-bloquant : l'onboarding continue sans logo  
}
```

Chapitre 11 — Base de données, migrations et Supabase

Modéliser et versionner sa base de données

Une base de données bien conçue, c'est la fondation de toute l'app. Social Seller utilise PostgreSQL (via Supabase) avec un schéma multi-tenant. Les migrations SQL versionnent chaque changement de schéma.

■ Analogie : Les migrations SQL, c'est comme Git pour la base de données

Git versionne ton code : tu peux revenir en arrière, voir qui a changé quoi, travailler à plusieurs. Les migrations font pareil pour ta base de données : chaque fichier SQL est un changement versionné. 001_initial_schema.sql → 002_... → 003_... — l'historique complet du schéma.

11.1 — Le schéma multi-tenant de Social Seller

```
-- Migration 001 : Tables de base

CREATE TABLE tenants (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  name TEXT NOT NULL,
  plan TEXT NOT NULL DEFAULT 'free',
  status TEXT NOT NULL DEFAULT 'active',
  owner_id UUID REFERENCES auth.users(id),
  country TEXT,
  currency TEXT,
  sector TEXT,
  logo_url TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  deleted_at TIMESTAMPTZ -- ← soft delete : jamais de DELETE réel
);

CREATE TABLE users (
  id UUID PRIMARY KEY REFERENCES auth.users(id),
  tenant_id UUID REFERENCES tenants(id),
  role TEXT NOT NULL DEFAULT 'merchant',
  phone TEXT,
  full_name TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Activer RLS sur toutes les tables (obligatoire)
```

```
ALTER TABLE tenants ENABLE ROW LEVEL SECURITY;
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
```

11.2 — Les triggers PostgreSQL

Un trigger est du code SQL qui s'exécute automatiquement quand quelque chose se passe en base de données. Dans Social Seller, un trigger crée la ligne `public.users` juste après la création d'un tenant :

```
-- Migration 003 : Trigger on_tenant_created
CREATE OR REPLACE FUNCTION on_tenant_created()
RETURNS TRIGGER
LANGUAGE plpgsql
SECURITY DEFINER -- ← s'exécute avec les droits admin, pas user
SET search_path = public
AS $$
BEGIN
INSERT INTO users (id, tenant_id, role, phone, full_name)
VALUES (
NEW.owner_id,
NEW.id,
'merchant',
(auth.jwt() -> 'user_metadata' ->> 'phone'),
(auth.jwt() -> 'user_metadata' ->> 'first_name') || ' ' ||
(auth.jwt() -> 'user_metadata' ->> 'last_name')
);
RETURN NEW;
END;
$$;

-- Lier le trigger à la table tenants
CREATE TRIGGER on_tenant_created
AFTER INSERT ON tenants
FOR EACH ROW EXECUTE FUNCTION on_tenant_created();
```

■ SECURITY DEFINER — avec grand pouvoir...

`SECURITY DEFINER` fait tourner la fonction avec les droits du créateur (superuser). Elle peut contourner RLS et écrire partout. Toujours ajouter `SET search_path = public` pour éviter les injections. Et toujours valider `auth.uid() IS NOT NULL` dedans.

Chapitre 12 — Row Level Security (RLS)

Sécuriser les données côté serveur

RLS est l'une des fonctionnalités les plus puissantes de PostgreSQL/Supabase. Elle garantit que chaque utilisateur ne peut voir et modifier QUE ses propres données, peu importe le code côté mobile. Même si un hacker modifie l'app, RLS bloque les accès non autorisés.

■ Analogie : RLS, c'est comme un videur avec une liste

Sans RLS : tu demandes 'donne-moi toutes les commandes' → tu les reçois toutes. Avec RLS : tu demandes 'donne-moi toutes les commandes' → le videur (PostgreSQL) vérifie ta carte d'identité (JWT) et te donne seulement VOS commandes. Le code mobile n'a pas besoin de filtrer — PostgreSQL le fait automatiquement.

12.1 — Les types de politiques

SELECT	Lecture (query)	Je lis seulement MON tenant
INSERT	Écriture (insert)	Je crée un tenant pour moi seulement
UPDATE	Modification (update)	Je modifie seulement MON tenant
DELETE	Suppression (delete)	Pas de DELETE → soft delete
ALL	Toutes les opérations	Shorthand pour 4 politiques en 1

12.2 — Les politiques de Social Seller

```
-- Policy SELECT : je lis seulement mon tenant
CREATE POLICY tenants_select ON tenants
FOR SELECT USING (id = current_tenant_id());
-- La fonction current_tenant_id() cherche le tenant de l'user connecté
CREATE OR REPLACE FUNCTION current_tenant_id()
RETURNS UUID LANGUAGE sql STABLE
AS $$ SELECT tenant_id FROM users WHERE id = auth.uid() $$;
-- Policy INSERT : je ne crée un tenant que pour moi
CREATE POLICY tenants_insert_self ON tenants
```

```
FOR INSERT WITH CHECK (owner_id = auth.uid());
-- Policy UPDATE : je modifie seulement mon tenant
CREATE POLICY tenants_update ON tenants
FOR UPDATE USING (id = current_tenant_id())
WITH CHECK (id = current_tenant_id());
```

12.3 — Le RPC pour contourner RLS en sécurité

Parfois on a besoin d'opérations qui ne peuvent pas passer par RLS directement (ex: lors de l'onboarding, créer le premier tenant). La solution : un RPC Postgres SECURITY DEFINER.

```
-- Migration 004 : RPC sécurisé pour créer le tenant initial
CREATE OR REPLACE FUNCTION create_initial_tenant(
  p_name TEXT, p_country TEXT, p_currency TEXT, p_sector TEXT
) RETURNS uuid
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public
AS $$
DECLARE v_tenant_id uuid;
BEGIN
  -- Vérifier que l'user est bien connecté
  IF auth.uid() IS NULL THEN
    RAISE EXCEPTION 'Not authenticated';
  END IF;

  -- Vérifier qu'il n'a pas déjà un tenant
  IF EXISTS (SELECT 1 FROM users WHERE id = auth.uid()) THEN
    RAISE EXCEPTION 'User already has a tenant';
  END IF;

  INSERT INTO tenants (name, owner_id, country, currency, sector)
  VALUES (p_name, auth.uid(), p_country, p_currency, p_sector)
  RETURNING id INTO v_tenant_id;

  RETURN v_tenant_id;
END;
$$;

GRANT EXECUTE ON FUNCTION create_initial_tenant TO authenticated;
```

Appel depuis l'app mobile :

```
// profile-setup.tsx – utiliser le RPC
const { data: tenantId, error } = await supabase.rpc('create_initial_tenant', {
  p_name: shopName.trim(),
  p_country: country,
  p_currency: COUNTRY_CURRENCY[country],
  p_sector: sector,
})
```

```
if (error) throw error
// tenantId contient l'UUID du tenant créé
```

Chapitre 13 — Navigation avec Expo Router

Tabs et authentification

La navigation dans Social Seller est organisée en deux groupes : (auth) pour les écrans non connectés, (tabs) pour les écrans connectés. Voici comment fonctionne la barre d'onglets et comment protéger les routes.

13.1 — La barre d'onglets (tabs)

```
// app/(tabs)/_layout.tsx – Les 5 onglets de Social Seller
import { Tabs } from 'expo-router'
import { Feather } from '@expo/vector-icons'

export default function TabLayout() {
  return (
    <Tabs screenOptions={{
      tabBarActiveTintColor: '#6366F1', // Indigo actif
      tabBarInactiveTintColor: '#94A3B8', // Slate inactif
      headerShown: false,
    }}>
      <Tabs.Screen name="index"
        options={{ title: 'Accueil',
          tabBarIcon: ({ color }) => <Feather name="home" color={color} size={22} /> }} />
      <Tabs.Screen name="inbox"
        options={{ title: 'Messages',
          tabBarIcon: ({ color }) => <Feather name="message-square" color={color} size={22} /> }} />
      <Tabs.Screen name="orders"
        options={{ title: 'Commandes',
          tabBarIcon: ({ color }) => <Feather name="shopping-cart" color={color} size={22} /> }} />
      <Tabs.Screen name="stock"
        options={{ title: 'Stock',
          tabBarIcon: ({ color }) => <Feather name="package" color={color} size={22} /> }} />
      <Tabs.Screen name="stats"
        options={{ title: 'Stats',
          tabBarIcon: ({ color }) => <Feather name="bar-chart-2" color={color} size={22} /> }} />
    </Tabs>
  )
}
```

13.2 — Les méthodes de navigation

<code>router.push(href)</code>	Ajoute au stack (retour possible)	'/(auth)/activation'
<code>router.replace(href)</code>	Remplace dans le stack (pas de retour)	'/(tabs)' après login
<code>router.back()</code>	Retour écran précédent	Bouton '← Retour'
	Redirect pendant le render	app/index.tsx auth guard
Link component	Lien déclaratif dans le JSX	'Déjà un compte ?'

■ Bug classique Expo Router : `router.replace()` dans `useEffect`

N'appelle jamais `router.replace()` dans un `useEffect` alors que le Stack navigator n'est pas encore monté. La navigation est ignorée silencieusement et tu te retrouves avec un écran blanc sans erreur. Utilise dans le render pour les redirections auth.

PARTIE 3

Bonnes pratiques

Sécurité, debug et collaboration

- Sécurité mobile
- Débugger comme un pro
- Git et GitHub
- Récap Sprint 1

Chapitre 14 — Sécurité mobile — Les règles d'or

Les règles de CLAUDE.md appliquées à Social Seller

Social Seller a un fichier CLAUDE.md qui liste toutes les règles de sécurité du projet. Ces règles ne sont pas optionnelles — elles protègent les données des marchands maliens et leur business. Voici les plus importantes expliquées.

■ ■ Jamais de secrets dans le code

Ne jamais hardcoder une clé API, un mot de passe ou un token dans le code source. Utilise toujours `process.env.EXPO_PUBLIC_*` pour les variables accessibles côté mobile et `process.env.*` (sans `EXPO_PUBLIC`) pour les secrets serveur uniquement.

■ ■ ■ expo-secure-store pour les tokens

Les tokens JWT ne doivent JAMAIS être stockés dans `AsyncStorage` (non chiffré) ou `localStorage`. Sur mobile, utilise `expo-secure-store` qui utilise le Keychain iOS et le Keystore Android. Sur web (dev uniquement), `localStorage` est acceptable mais doit être retiré en production.

■ ■ tenant_id sur chaque requête Supabase

Chaque requête Supabase doit être filtrée par `tenant_id`. RLS le fait automatiquement via `current_tenant_id()`, mais c'est une règle mentale importante : jamais de `SELECT * FROM orders` sans scope tenant.

■ ■ ■ Soft delete uniquement

Jamais de `DELETE` en base de données. Toutes les tables ont une colonne `deleted_at`. Pour supprimer : `UPDATE table SET deleted_at = NOW() WHERE id = X`. Ça permet de récupérer des données supprimées par erreur.

■ ■ ■ Pas de console.log avec données sensibles

`console.log('user:', user)` en production expose l'ID, l'email, les métadonnées. Log uniquement des confirmations non-sensibles : `'login: success'`, `'tenant: created'`. Avant tout build production, cherche et supprime tous les `console.log` sensibles.

■ ■ RLS activé sur TOUTES les tables

Ne jamais créer une table sans immédiatement activer RLS et créer les politiques. Une table sans RLS = toutes les données lisibles par n'importe quel utilisateur authentifié.

Chapitre 15 — Débugger comme un pro

Lire les erreurs et les résoudre systématiquement

Débugger, c'est une compétence qui s'apprend. Voici les erreurs rencontrées pendant le Sprint 1 de Social Seller et les méthodes pour les diagnostiquer.

■■ Erreur : Écran blanc sans erreur

Cause : `router.replace()` appelé avant que le Stack navigator soit monté. Méthode : ajouter un écran de chargement visible pour confirmer que le composant rend. Si ça rend mais ne navigue pas, c'est le timing du router. Fix : utiliser dans le render plutôt que `router.replace()` dans `useEffect`.

■ Erreur : new row violates row-level security policy

Cause : le JWT de l'utilisateur n'est pas envoyé avec la requête Supabase, donc `auth.uid()` retourne null et la policy échoue. Fix : utiliser un RPC SECURITY DEFINER pour les opérations d'onboarding où l'état de session peut être incertain.

■■ Erreur : ExpoSecureStore.default.getValueWithKeyAsync is not a function

Cause : expo-secure-store est un module natif — il n'existe pas sur web. Fix : créer un storage adapter platform-aware qui utilise `localStorage` sur web et expo-secure-store sur native.

■ Erreur : Incompatible SDK version (Expo Go)

Cause : la version SDK du projet ne correspond pas à celle supportée par Expo Go installé. Fix : downgrader le projet au SDK supporté par Expo Go (`npm expo install --fix --legacy-peer-deps`). Aucun impact code.

■■ Erreur : Email rate limit exceeded

Cause : Supabase free tier limite les appels d'inscription. Fix court terme : attendre 1h. Fix long terme : configurer un SMTP custom (Resend.com gratuit) dans Supabase Dashboard → Authentication → SMTP.

■ Erreur : Cannot read properties of undefined (reading 'showActionSheetWithOptions')

Cause : `ActionSheetIOS` est iOS uniquement — `undefined` sur web et Android. Fix : condition `Platform.OS === 'ios'` (pas `!== 'android'`) pour inclure web dans la branche `Picker`.

15.1 — La méthode de debug systématique

1 Lire l'erreur entière

La première ligne dit QUOI, le call stack dit OÙ. Ne pas ignorer le call stack.

2 Identifier la couche

Erreur dans le code (logic) ? Dans un package (import) ? Dans Supabase (RLS/SQL) ? Dans le réseau ?

3 Isoler

Commenter du code jusqu'à ce que l'erreur disparaisse. Le dernier bloc commenté est le coupable.

4 Lire la doc

90% des erreurs Expo/Supabase ont une issue GitHub ou une réponse StackOverflow. Cherche le message d'erreur exact.

5 Ajouter des logs

`console.log` à chaque étape clé pour voir jusqu'où le code arrive. Retirer les logs après.

Chapitre 16 — Git et GitHub en pratique

Versionner son travail comme un professionnel

Git est le système de contrôle de version utilisé par 95% des développeurs dans le monde. Il te permet de sauvegarder chaque état de ton code, de collaborer, et de revenir en arrière si quelque chose casse. GitHub héberge ton code en ligne.

16.1 — Les commandes Git essentielles

git init	Initialiser un repo Git dans le dossier courant
git status	Voir les fichiers modifiés depuis le dernier commit
git add .	Ajouter TOUS les fichiers modifiés au commit
git add fichier.tsx	Ajouter UN fichier spécifique
git commit -m 'message'	Sauvegarder les changements avec un message
git log --oneline	Voir l'historique des commits
git push origin main	Envoyer les commits vers GitHub
git pull origin main	Récupérer les changements depuis GitHub
git checkout -b feature/nom	Créer une nouvelle branche

16.2 — Le message de commit du Sprint 1

Un bon message de commit suit la convention 'feat:' / 'fix:' / 'chore:'. Voici le commit du Sprint 1 de Social Seller :

```
feat: Sprint 1 – auth screens (login, register, activation, profile-setup),  
navigation tabs, Supabase schema migrations 001-004, RPC create_initial_tenant  
# 29 fichiers, 12138 insertions(+)  
# Hash: 471dbea
```

■ Le .gitignore protège tes secrets

Avant tout commit, vérifie que `.gitignore` contient : `.env.local` `node_modules/` `.expo/` Si `.env.local` est commité par erreur, change IMMÉDIATEMENT ta clé Supabase (Supabase Dashboard → Settings → API → Regenerate). Une clé exposée = toutes tes données compromises.

Chapitre 17 — Récap Sprint 1 et préparation Sprint 2

Bilan et prochaines étapes

Le Sprint 1 est terminé et validé. Voici ce qui a été construit, les leçons apprises, et ce qui attend au Sprint 2.

17.1 — Ce qui est livré

■ 5 écrans d'authentification	Login, Register, Activation WhatsApp, Profile Setup, Tabs
■ Navigation Expo Router v4	Groupes (auth) et (tabs), auth guard dans app/index.tsx
■ Supabase Auth	Email provider avec pattern +223XXXXXXXXX@socialseller.app
■ 4 migrations SQL	Schéma multi-tenant, RLS, triggers, RPC create_initial_tenant
■ Stockage sécurisé	expo-secure-store sur native, localStorage sur web (dev)
■ lib/constants.ts	EMAIL_DOMAIN, COUNTRY_CODE, PHONE_DIGITS_LENGTH centralisés
■ Premier commit Git	471dbea — 29 fichiers, repo GitHub privé MedMoh223/social-seller

17.2 — Les leçons du Sprint 1

- Expo Router : Redirect > router.replace() pour les redirections auth
- RLS + onboarding : utiliser un RPC SECURITY DEFINER pour le premier INSERT
- Platform-aware : toujours vérifier Platform.OS === 'ios' (inclut web dans sinon)
- expo-secure-store : natif uniquement, jamais utilisé sur web
- SDK Expo Go : vérifier la compatibilité avant de choisir la version SDK
- Email rate limit : configurer SMTP custom dès le début du projet
- .gitignore : toujours vérifier avant le premier commit

17.3 — Ce qui attend au Sprint 2

■ Backend Railway	API Node.js pour l'activation WhatsApp, les webhooks Facebook/TikTok
■ SMTP custom	Resend.com pour lever le rate limit Supabase en développement
■ Development Build	Build natif personnalisé pour tester sur vrai device sans Expo Go
■ Inbox	Écran des messages centralisés WhatsApp + Facebook + TikTok
■ Orders	Gestion des commandes avec statuts
■ Stock	Gestion de l'inventaire produits
■ Auth guard complet	Vérifier que l'user a un tenant avant de router vers tabs

"Le code le plus sécurisé est celui qu'on n'a pas écrit — " mais celui qu'on a écrit doit l'être à fond.

À toi de jouer pour le Sprint 2. ■